

Contents

1	Introduction	2
1.1	Instantiation Time	2
1.2	Runtime Representation	2
1.3	Reflection and Runtime Type Information	3
1.4	Type Safety	3
1.5	Performance	3
1.6	Summary	4
2	Instantiation Time	5
2.1	Instantiation Time in C++ Templates	5
2.1.1	Template Definition	5
2.1.2	Instantiation Trigger	6
2.1.3	Template Argument Substitution	6
2.1.4	Semantic Analysis of the Specialization	7
2.1.5	Intermediate Representation and Code Generation	7
2.1.6	Completion of the Instantiation Process	8
2.2	Analysis of the Algorithms	8

Chapter 1

Introduction

Parametric polymorphism provides a language-independent abstraction, its implementation varies considerably among programming languages and compiler architectures.

The principal design choice concerns how generic types are represented after compilation and when concrete type instantiations are produced.

These implementation strategies influence several aspects of the execution model, including runtime representation, reflection capabilities, performance, and binary size.

The following dimensions constitute the primary axes along which implementations of parametric polymorphism are typically compared.

1.1 Instantiation Time

A first distinguishing characteristic is the stage at which generic definitions are instantiated.

Some implementations perform instantiation entirely during compilation.

In this approach, each concrete use of a generic function or data structure is expanded into a specialized implementation before executable code is generated.

Since all specializations already exist in the final binary, runtime execution incurs no additional generic instantiation cost.

Other implementations compile generic definitions only once.

Generic type parameters are verified during type checking and subsequently removed or replaced by a common representation.

Consequently, no additional executable code is generated for different type arguments, and all instantiations share the same compiled implementation.

1.2 Runtime Representation

Implementations also differ in how generic types are represented after compilation.

Type erasure removes generic type parameters, producing a homogeneous representation where different instantiations share the same executable code.

The runtime system therefore treats different generic instantiations as equivalent representations.

Conversely, *reified generics* preserve concrete type information by generating distinct specialized code for each instantiated type.

Each generic instantiation may therefore possess its own runtime representation, memory layout, and executable instructions.

1.3 Reflection and Runtime Type Information

The availability of generic type information at runtime depends directly on the chosen representation strategy.

When type erasure is employed, generic parameters are generally unavailable after compilation.

Reflection mechanisms therefore provide limited access to instantiated generic types because most type information has been discarded.

Conversely, implementations based on reified generics preserve concrete type information throughout compilation.

Runtime reflection or runtime type identification (RTTI) may therefore recover the actual instantiated types, enabling operations such as generic type inspection, specialized serialization, and runtime metadata analysis.

1.4 Type Safety

Both implementation strategies preserve static type safety, although they achieve this objective differently.

In implementations based on type erasure, generic correctness is verified entirely during compilation.

Although type parameters disappear from the generated executable, compiler-inserted casts preserve the guarantees established by the static type system.

Implementations based on reified generics retain complete type information throughout compilation and generate type-specific implementations directly.

Since each specialization corresponds to a concrete type, correctness is preserved without requiring erased representations or implicit runtime casts.

1.5 Performance

The implementation strategy significantly influences both compilation and execution performance.

Homogeneous implementations generally produce smaller binaries and reduce compilation time because generic definitions are compiled only once.

However, the shared representation may require additional indirections, boxing, or runtime casts, limiting optimization opportunities.

Heterogeneous implementations incur greater compilation cost and larger executables because every instantiated type generates a distinct implementation.

Nevertheless, the compiler can aggressively optimize each specialization, exploiting concrete type information for improved instruction selection, memory layout, inlining, and register allocation.

Consequently, runtime performance often approaches that of manually specialized code.

1.6 Summary

The implementation of parametric polymorphism is primarily characterized by five orthogonal dimensions:

1. the stage at which generic instantiation occurs;
2. the runtime representation of generic types;
3. the availability of runtime type information and reflection;
4. the trade-off between compilation cost, binary size, and execution performance;
5. the mechanisms through which static type safety is preserved.

These dimensions form the basis for comparing homogeneous implementations based on type erasure with heterogeneous implementations based on reification or monomorphization.

Chapter 2

Instantiation Time

2.1 Instantiation Time in C++ Templates

C++ implements parametric polymorphism through the *template* mechanism, which adopts a purely compile-time instantiation model.

Unlike languages that compile a generic definition only once, a C++ template is not itself an executable entity. Instead, it represents a parameterized program definition that serves as a blueprint from which the compiler generates independent concrete implementations.

From the perspective of instantiation time, the defining characteristic of C++ is that executable code is generated only after the compiler knows the complete set of template arguments.

Consequently, all template instantiations are resolved entirely during static compilation, and no template instantiation mechanism exists during program execution.

The complete instantiation process can be understood by following the template through the compilation pipeline.

2.1.1 Template Definition

When the compiler encounters a template definition, it first performs the ordinary front-end compilation stages, including lexical analysis, parsing, and semantic analysis that can be performed without knowledge of future template arguments.

For example:

```
1  template<typename T>
2  T get_max(T a, T b)
3  {
4      return (a > b) ? a : b;
5  }
```

The compiler parses the template syntax, builds the corresponding Abstract Syntax Tree (AST), checks syntactic correctness, and stores an internal parameterized representation.

At this stage, the compiler does not generate executable machine code.

The reason is straightforward: the compiler still ignores the concrete meaning of the template parameter T , and therefore cannot determine the exact operations, object layout, calling convention, or generated instructions required by the final implementation.

Consequently, the template definition remains a generic blueprint waiting for future instantiation.

2.1.2 Instantiation Trigger

The instantiation process begins only when the compiler encounters a concrete use of the template, either through an explicit specification of the template argument or through implicit type deduction.

Example.

Consider the following template invocations:

```
int i = get_max(3, 7);  
double d = get_max<double>(2.5, 4.8);
```

The first invocation relies on template argument deduction:

$$T = \textit{int}$$

whereas the second invocation explicitly specifies the template argument:

$$T = \textit{double}$$

Therefore, the compiler generates two different template instantiations: `get_max<int>` and `get_max<double>`

Each unique set of template arguments constitutes a distinct template instantiation. Instantiation may occur through:

- **implicit instantiation**, automatically performed whenever a specialization is required by the program;
- **explicit instantiation**, requested directly by the programmer through an explicit instantiation declaration or definition;
- **explicit specialization**, where an entirely different implementation is provided for a specific combination of template arguments.

Regardless of how instantiation is initiated, executable code generation begins only after the complete set of template arguments is known.

2.1.3 Template Argument Substitution

Once instantiation has been triggered, the compiler performs template argument substitution.

During this phase, every formal template parameter is replaced by its corresponding concrete argument.

From previous example.

$$T \rightarrow int$$

produces the specialization:

```
1 int get_max(int a, int b)
2 {
3     return (a > b) ? a : b;
4 }
```

whereas:

$$T \rightarrow double$$

produces:

```
1 double get_max<double>(double a, double b)
2 {
3     return (a > b) ? a : b;
4 }
```

Conceptually, the compiler now treats each specialization as if it had been written explicitly by the programmer.

From this point onward, the specialization behaves as an ordinary non-template function.

2.1.4 Semantic Analysis of the Specialization

Instantiation does not immediately produce executable code.

Instead, each generated specialization undergoes the normal semantic analysis performed for ordinary C++ program entities.

Depending on the specialization, the compiler performs activities such as: type checking, overload resolution, name lookup, constraint verification for constrained templates, constant expression evaluation, instantiation of nested templates and verification of language rules required by the selected template arguments.

This step is fundamental because many semantic properties cannot be verified before template parameters have been substituted.

Consequently, a template definition that is syntactically correct may still produce compilation errors if one of its concrete instantiations violates the requirements imposed by the template body.

2.1.5 Intermediate Representation and Code Generation

Once semantic analysis has successfully completed, the specialization enters the compiler back-end.

The compiler generates an Intermediate Representation (IR) specifically for the instantiated specialization.

Subsequently, the specialization undergoes the standard optimization pipeline, which may include: constant propagation, dead-code elimination, function inlining, loop optimizations, register allocation, instruction selection and target-specific optimizations.

Finally, the compiler emits native machine code corresponding exclusively to that specialization.

Importantly, this process is repeated independently for every distinct template instantiation.

Consequently, each specialization owns an independent executable implementation. Infact, the compiler does not produce a single shared executable representation capable of serving multiple template arguments.

2.1.6 Completion of the Instantiation Process

The instantiation process terminates entirely before linking.

By the time object files are generated, every template specialization required by the translation unit already exists as ordinary compiled code.

After linking, the final executable therefore contains every specialization required by the program.

From the perspective of executable code generation, the lifecycle of a C++ template can be summarized as follows:

1. The compiler parses the template definition.
2. The template is stored as a parameterized internal representation.
3. A concrete template use triggers instantiation.
4. Template parameters are substituted with concrete arguments.
5. The generated specialization undergoes complete semantic analysis.
6. The compiler produces a specialization-specific intermediate representation.
7. Optimization passes are applied.
8. Native machine code is emitted.
9. The specialization is included in the final executable before program execution begins.

Therefore, the defining characteristic of C++ parametric polymorphism is that the entire instantiation process is completed during static compilation.

When execution begins, no template instantiation mechanism remains active.

The runtime simply executes machine code that has already been generated for every required specialization during compilation.

2.2 Analysis of the Algorithms

You can use sections to divide your chapters into smaller, readable chunks.